Control Abstraction of Divide and Conquer

- A control abstraction is a procedure whose flow of control is clear but whose primary operations are specified by other procedures whose precise meanings are left undefined.
- The control abstraction for divide and conquer technique is DANDC(P), where P is the problem to be solved.

DANDC (P)

{

if SMALL (P) then return S (P);

else

{

Divide P into smaller instances P_1, P_2, …. P_k, k≥1

Apply DANDC to each of these sub problems;

Return (COMBINE (DANDC (P_1), DANDC (P_2),…., DANDC (P_k)) ;

}

}

- ✓ SMALL(P) is a Boolean valued function which determines whether the input size is small enough so that the answer can be computed without splitting.
- ✓ If this is so function 'S' is invoked otherwise, the problem 'P' is divided into smaller sub problems.
- ✓ These sub problems P_1, P_2, . . . , P_k are solved by recursive application of DANDC.

Control Abstraction of Divide and Conquer

- A control abstraction is a procedure whose flow of control is clear but whose primary operations are specified by other procedures whose precise meanings are left undefined.
- The control abstraction for divide and conquer technique is DANDC(P), where P is the problem to be solved.

DANDC (P)

{

if SMALL (P) then return S (P);

else

{

Divide P into smaller instances P_1, P_2, …. P_k, k≥1

Apply DANDC to each of these sub problems;

Return (COMBINE (DANDC (P_1), DANDC (P_2),…., DANDC (P_k)) ;

}

}

- ✓ SMALL(P) is a Boolean valued function which determines whether the input size is small enough so that the answer can be computed without splitting.
- ✓ If this is so function 'S' is invoked otherwise, the problem 'P' is divided into smaller sub problems.
- ✓ These sub problems P_1, P_2, . . . , P_k are solved by recursive application of DANDC.